



Algorithms, Design, and Analysis

Dr. Allam Ameen

Assistant Professor

Helwan International Technological University (HITU)

Electronics Research Institute, Microstrip Department

Spring 2026

Lecture Outcomes

- Learn how to represent any algorithm with different techniques.

Algorithms Representation

Representing Algorithms تمثيل الخوارزميات

- How to Express Algorithms in a Clear, Precise, and Unambiguous manner?
- How to Represent Algorithms?
 - In Terms of Natural Language
 - In Terms of Formal Programming Language
 - In Terms of Pseudocode
 - Flowcharts

Representing Algorithms تمثيل الخوارزميات

(1) In Terms of Natural Language

Assume the first item is largest. Look at each of the remaining items in the list and if it is larger than the largest item so far, make a note of it. The last noted item is the largest in the list when the process is complete.

Advantages

- Familiar

Disadvantages

- Verbose
- Imprecise -> Ambiguity
- Rarely used for complex or technical algorithms

تمثيل الخوارزميات Representing Algorithms

(2) In Terms of Formal Programming Language

```
#Sum of natural numbers up to num
num = int(input("enter a number"))
if num < 0:
    print("Enter a positive number")
else:
    sum = 0
    while(num > 0):
        sum += num
        num -= 1
    print("The sum is", sum)
```

Advantages

- **Precise → Unambiguous**
- **Will ultimately program in these languages**

Disadvantages

- **Can be too low-level for algorithm design.**
- **It has syntactic details which are not important at the algorithm design phase.**
- **Not familiar for the person who is not interested in this programming language.**

تمثيل الخوارزميات Representing Algorithms

(3) In Terms of Pseudocode

1. Get array list and its size n
2. Assign max = list[1]
3. For i = 2 to n Do
 1. IF (list[i] > max) THEN
 1. max = list[i]
 2. End If
4. End For
5. Display max

Advantages

- A middle-ground compromise.
- Resembles many popular programming languages.
- Relatively free of grammatical rules.
- Only well-defined statements are included “A Programming language without details”.

Disadvantages

- Compared with flowchart it is difficult to understand the program logic

Pseudocode

- Consists of **natural language-like statements** that precisely describe the steps of an algorithm or program.
- Statements describe actions.
- Focuses on the **logic of the algorithm** or program.
avoids language-specific elements.
- Written at **a level** so that the desired programming code can be generated almost automatically from each statement.
- Steps are numbered. Subordinate numbers and/or **indentation** are used for dependent statements in selection and repetition structures.

What is a Pseudocode?

Pseudocode (derived from pseudo and code) is a compact and informal high-level description of a computer programming algorithm that uses the structural conventions of some programming language, but typically omits details that are not essential for the understanding of the algorithm, such as subroutines, variable declarations and system-specific code.

SOME NOTES

- No standard pseudo code syntax exists.
- A program in pseudocode is not an executable program.
- Flowcharts can be thought of as a graphical alternative to pseudocode.

Pseudocode Language Constructs

- Computation/Assignment
 - **Compute** var1 as the sum of x and y
 - **Assign** expression to var2
 - **Increment** counter1
- Input/Output
 - Input: **Get** var1, var2, ...
 - Output: **Display** var1, var2, ...

Pseudocode Language Constructs

Selection

Single-Selection IF

1. **IF** *condition* **THEN** (IF condition is true, then do subordinate statement 1, etc. If condition is false, then skip statements)
 - 1.1 statement 1
 - 1.2 etc.

Double-Selection IF

2. **IF** *condition* **THEN** (IF condition is true, then do subordinate statement 1, etc. If condition is false, then skip statements and execute statements under ELSE)
 - 2.1 statement 1
 - 2.2 etc.
3. **ELSE** (else if condition is not true, then do subordinate statement 2, etc.)
 - 3.1 statement 2
 - 3.2 statement 3
4. **SWITCH** *expression* **TO**
 - 4.1 case 1: action1
 - 4.2 case 2: action2
 - 4.3 etc.
 - 4.4 default: actionx



Pseudocode Language Constructs

Repetition

5. **WHILE** *condition* (while condition is true, then do subordinate statements)

5.1 statement 1

5.2 etc.

DO – WHILE structure (like WHILE, but tests condition at the end of the loop. Thus, statements in the structure will always be executed at least once.)

6. **DO**

6.1 statement 1

6.2 etc.

7. **WHILE** *condition*

FOR structure (a specialized version of WHILE for repeating execution of statements a specific number of times)

8. **FOR** *bounds on repetition*

8.1 statement 1

8.2 etc.

Pseudocode Example

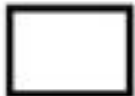
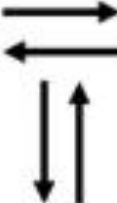
- Express an algorithm to get two numbers from the user (dividend and divisor), testing to make sure that the divisor number is not zero, and displaying their quotient using pseudocode.

Solution

1. Get dividend, divisor
2. IF divisor = 0 THEN
 - 1) Display error message "divisor must be non-zero"
 - 2) Exit algorithm
3. Compute quotient = dividend/divisor
4. Display dividend, divisor, quotient

(4) Flowcharts

- A **flowchart** is a graphical representation of an algorithm. Once the flowchart is drawn, it becomes easy to write the program in any high-level language.

Symbol	Name/Meaning	Symbol	Meaning
	<u>Process</u> – Any type of internal operation: data transformation, data movement, logic operation, etc.		<u>Connector</u> – connects sections of the flowchart, so that the diagram can maintain a smooth, linear flow
	<u>Input/Output</u> – input or output of data		<u>Terminal</u> – indicates start or end of the program or algorithm
	<u>Decision</u> – evaluates a condition or statement and branches depending on whether the evaluation is true or false		<u>Flow lines</u> – arrows that indicate the direction of the progression of the program

Other Flowchart symbols



Terminator

Indicates the beginning or end of a program flow in your diagram.



Process

Indicates any processing function.



Decision

Indicates a decision point between two or more paths in a flowchart.



Delay

Indicates a delay in the process.



Data

Can represent any type of data in a flowchart.



Document

Indicates data that can be read by people, such as printed output.



Multiple documents

Indicates multiple documents.



Subroutine

Indicates a predefined (named) process, such as a subroutine or a module.



Preparation

Indicates a modification to a process, such as setting a switch or initializing a routine.



Display

Indicates data that is displayed for people to read, such as data on a monitor or projector screen.



Manual input

Indicates any operation that is performed manually (by a person).



Manual loop

Indicates a sequence of commands that will continue to repeat until stopped manually.



Loop limit

Indicates the start of a loop. Flip the shape vertically to indicate the end of a loop.



Stored data

Indicates any type of stored data.



Connector

Indicates an inspection point.



Off-page connector

Use this shape to create a cross-reference and hyperlink from a process on one page to a process on another page.



Off-page connector



Off-page connector



Off-page connector



Or

Logical OR



Summing junction

Logical AND



Collate

Indicates a step that organizes data into a standard format.



Sort

Indicates a step that organizes items list sequentially.



Merge

Indicates a step that combines multiple sets into one.



Database

Indicates a list of information with a standard structure that allows for searching and sorting.



Internal storage

Indicates an internal storage device.

Flowcharts

- Benefits of Flowcharts

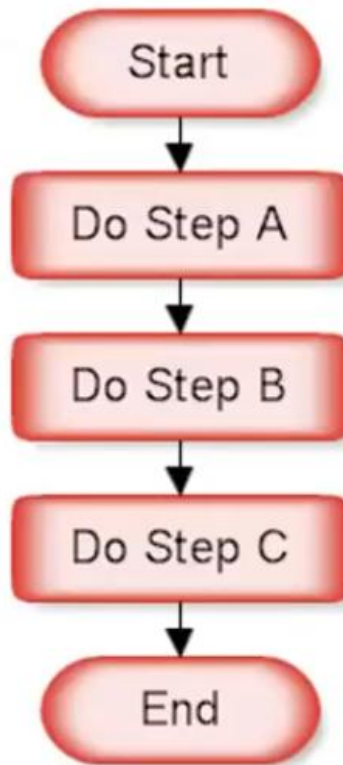
- Makes Logic Clear
- Effective Analysis
- Easy to Code and Test

- Limits of Flowcharts

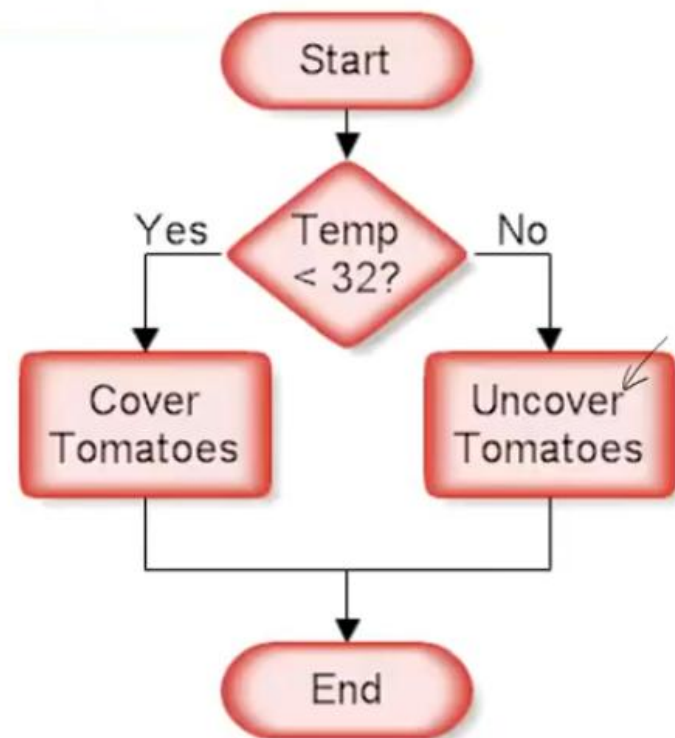
- Difficult to use for large Programs
- Difficult to Modify

Flowchart Constructs

Sequence



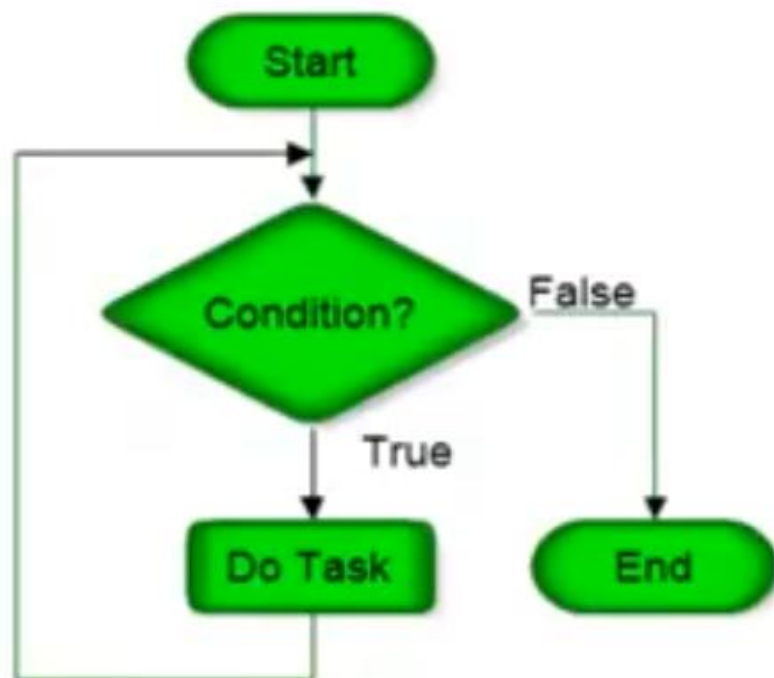
Decision



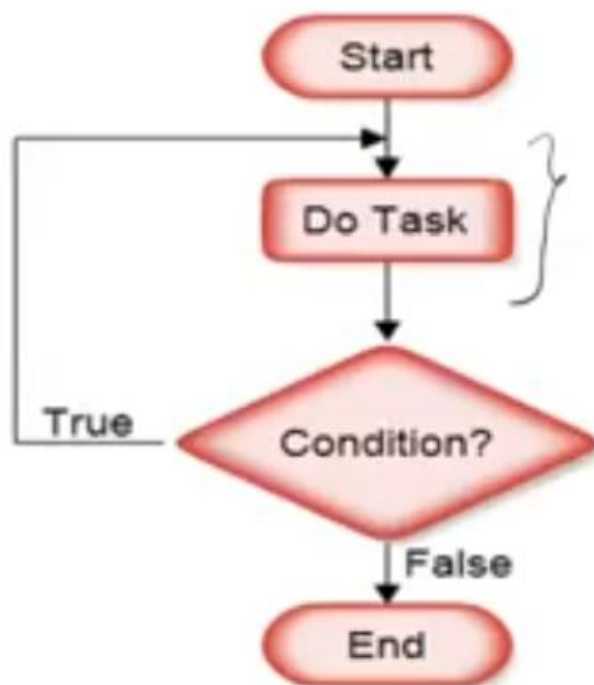
Flowchart Constructs

Loop

While Loop



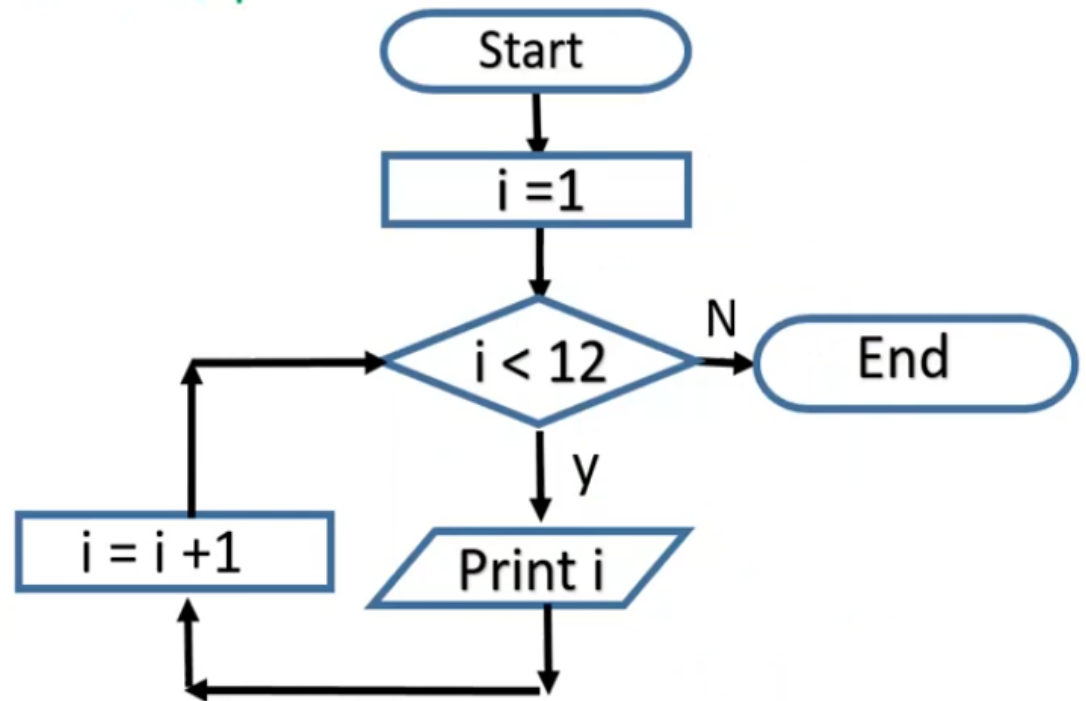
Do While Loop



Flowchart Constructs

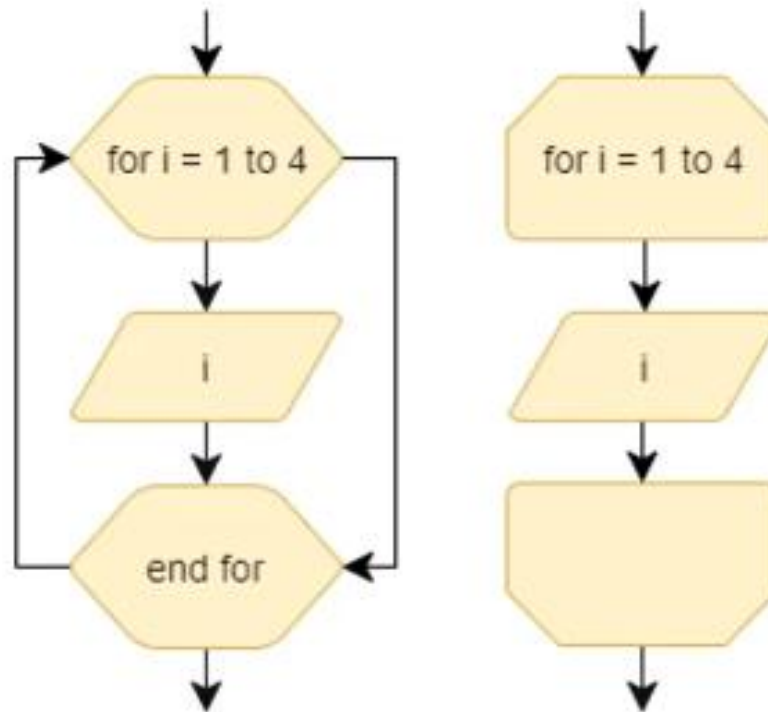
for Loops

```
for i in range (1,12):  
    print(i)
```



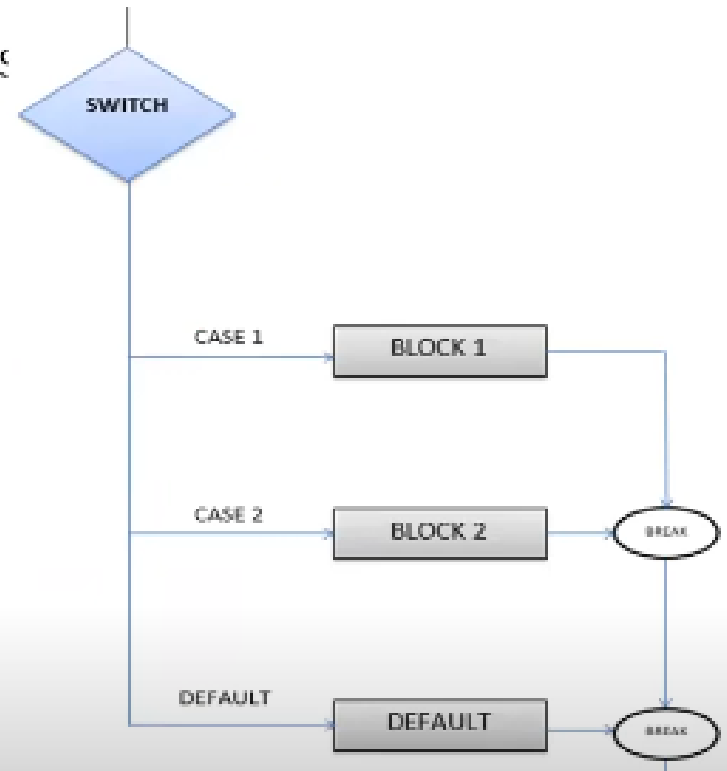
Flowchart Constructs

- loops are often drawn by the decision symbol. In some diagrams, you can encounter a notation using the Preparation symbol or the Loop limit symbol.

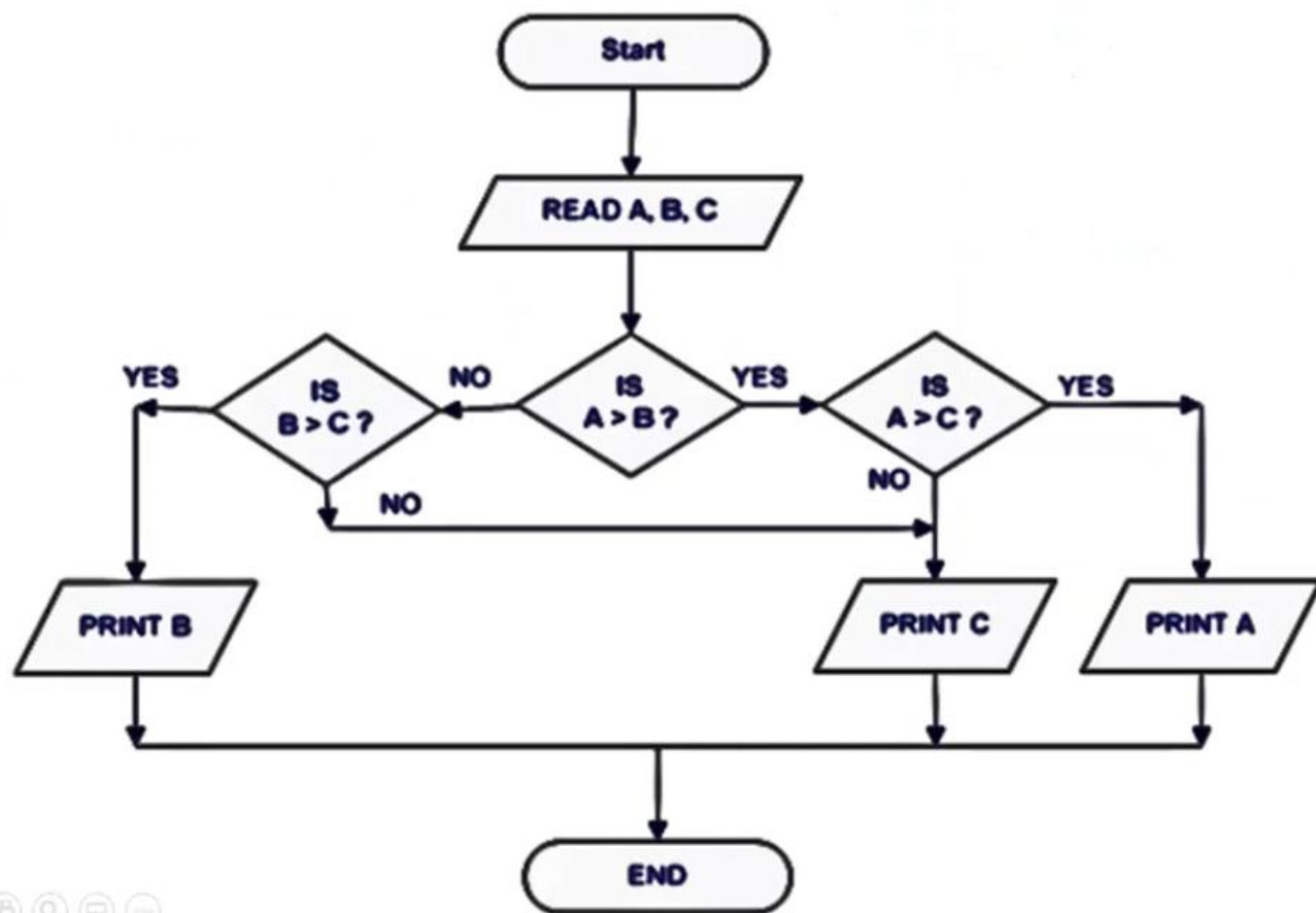


Flowchart Constructs

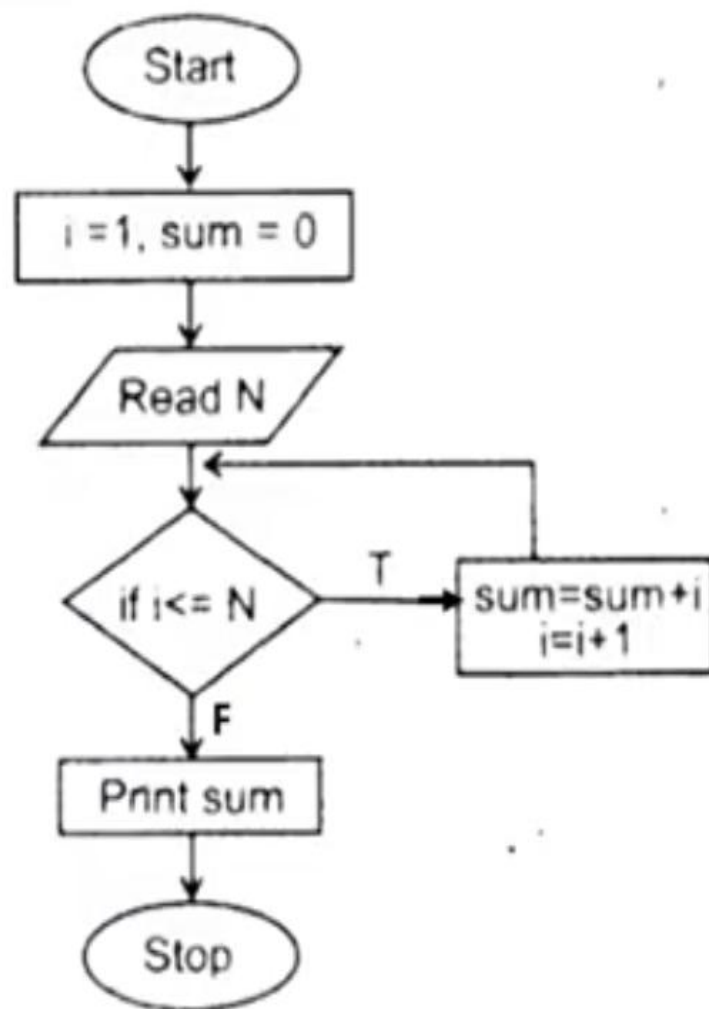
- ❑ Switch Case tests the value of a variable and compares it with multiple cases. Once the case match is found, a block of statements associated with that case is executed. Each case in a block of a switch has a different name/number which is referred to as an identifier. The value provided by the user is compared with all the cases inside the switch block until the match is found.



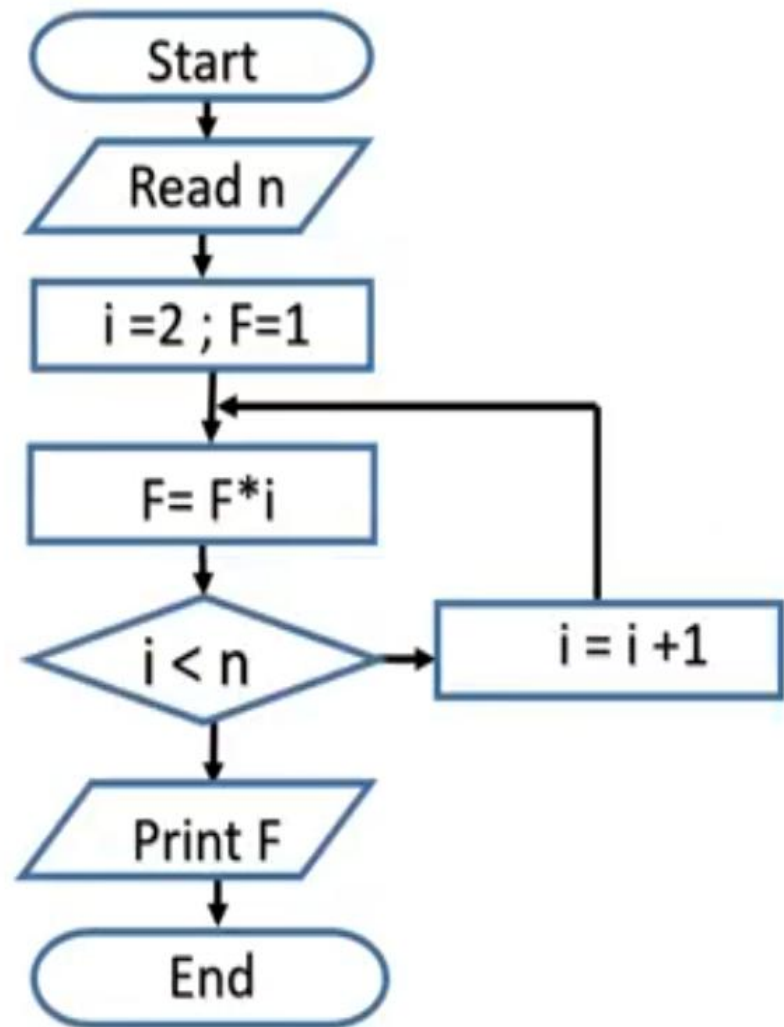
Example 1: Draw a flowchart to find the largest of three numbers A, B, and C.



Example 2: Sum of the first N numbers



Example 3: Flowchart to find $n!$



Example 4: Draw a flowchart to Find all the roots of a quadratic equation

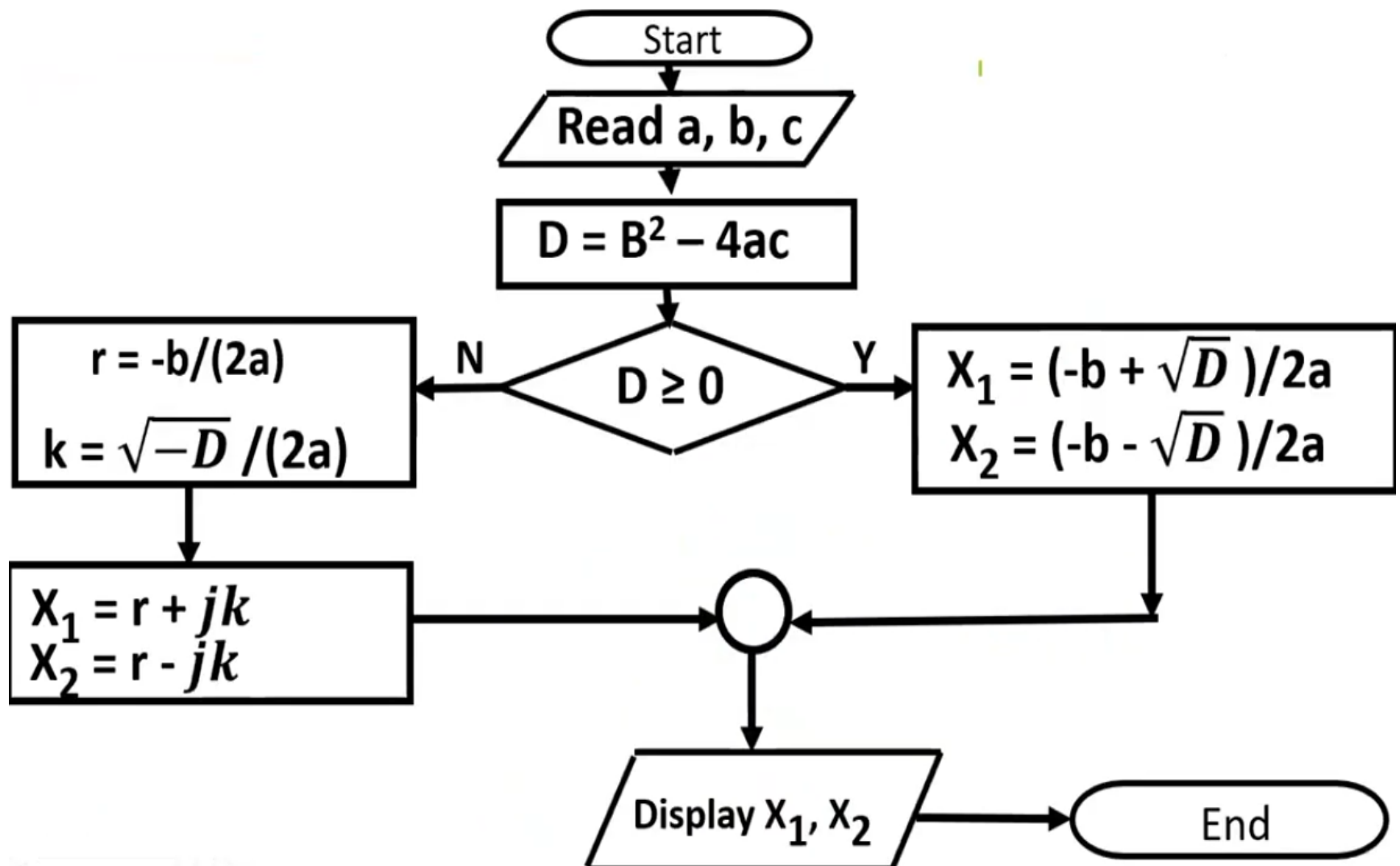
$$ax^2 + bx + c = 0$$

$$D = b^2 - 4ac$$

$$x_{1,2} = \frac{-b \pm \sqrt{D}}{2a}$$

$$\left\{ \begin{array}{l} D=0 \quad x_1 = x_2 = \frac{-b}{2a} \\ D>0 \quad x_1 = \frac{-b + \sqrt{D}}{2a} \\ \quad \quad x_2 = \frac{-b - \sqrt{D}}{2a} \end{array} \right\}$$

$$D < 0$$
$$x_{1,2} = \frac{-b}{2a} \pm \frac{j\sqrt{-D}}{2a}$$
$$x_1 = r + jk$$
$$x_2 = r - jk$$



Any
Questions ?

